

SSAFY Vanilla

SSAFY: Surprise-Sensitive Adaptive Framework for Your Answers

1. Overview

SSAFY (Surprise-Sensitive Adaptive Framework for Your Answers)는 이미지와 자연어 질문, 네 개의 선택지를 입력받아 정답을 **a**, **b**, **c**, **d** 중 하나로 예측하는 **Visual Question Answering (VQA)** 기반 다중 선택 모델이다.

SSAFY는 대규모 Vision-Language Model(VLM)을 처음부터 학습하지 않고, 사전학습된 Qwen-VL 계열 모델에 **4-bit quantization**과 **LoRA(Low-Rank Adaptation)**를 결합한 QLoRA 방식으로 특정 VQA 도메인에 적응시키는 것을 목표로 한다.

전체 모델의 핵심 설계는 다음 네 가지 원칙에 기반한다.

1. 사전학습된 멀티모달 모델의 시각·언어 표현 능력을 최대한 유지한다.
2. 전체 파라미터가 아닌 언어 모델의 일부 projection layer만 LoRA로 학습한다.
3. 학습 과정에서는 정답에 해당하는 단 하나의 assistant token에만 loss를 부여한다.
4. 추론 과정에서는 자유 생성 대신 **a**, **b**, **c**, **d** 네 후보 token의 next-token logit을 직접 비교한다.

따라서 SSAFY는 일반적인 생성형 VQA를 **제약된 4지선다 분류 문제로 변환**하여 생성 오류를 최소화하면서도 VLM의 이미지·텍스트 추론 능력을 활용한다.

2. Problem Formulation

각 데이터 샘플은 다음과 같이 정의된다.

$$x_i = (I_i, Q_i, O_i)$$

여기서

- (I_i) : 입력 이미지
- (Q_i) : 이미지에 대한 자연어 질문

- $(O_i = \{o_a, o_b, o_c, o_d\})$: 네 개의 선택지

이며 학습 데이터에는 추가적으로 정답

$$y_i \in a, b, c, d$$

가 존재한다.

모델의 목표는 다음 함수를 학습하는 것이다.

$$f(I_i, Q_i, O_i) \rightarrow y_i$$

SSAFY는 일반적인 텍스트 생성 방식으로 답을 생성하지 않고, assistant 응답이 시작되는 위치에서 네 선택지 token의 conditional score를 계산한다.

$$s_k = z_{t,k}, \quad k \in a, b, c, d$$

여기서 (z_t) 는 마지막 prompt token 뒤에서 모델이 출력하는 vocabulary logit vector이다.

최종 예측은 다음과 같다.

$$\arg \max_{k \in a, b, c, d} s_k$$

이 구조를 통해 모델은 긴 자연어를 생성할 필요 없이 단일 forward pass의 next-token distribution만으로 객관식 문제를 해결할 수 있다.

3. Dataset

SSAFY는 세 종류의 데이터셋을 사용하도록 설계되었다.

3.1 Training Dataset

`train.csv` 는 실제 학습에 사용되는 labeled dataset이다.

주요 필드는 다음과 같다.

Field	Description
<code>id</code>	샘플 식별자
<code>path</code>	이미지 경로
<code>question</code>	이미지 기반 질문

Field	Description
a	선택지 A
b	선택지 B
c	선택지 C
d	선택지 D
answer	정답 a/b/c/d

전체 labeled dataset은 학습 시작 전에 stratified split을 통해 training set과 validation set으로 분리된다.

```

Labeled Train
├── 90% → Training Pool
└── 10% → Fixed Validation Set

```

분할에는 정답 label을 기준으로 stratification을 적용한다.

이를 통해 a, b, c, d의 비율이 training과 validation에서 크게 달라지는 것을 방지한다.

3.2 Development Dataset

dev.csv는 정답 하나 대신 다수의 인간 응답을 포함한다.

```

answer1
answer2
answer3
answer4
answer5

```

현재 SSAFY 기본 구현에서는 이 다중 annotator 응답을 학습 loss에 직접 사용하지 않는다.

따라서 dev 데이터는 향후 다음과 같은 확장을 위한 잠재적인 정보원으로 볼 수 있다.

- 인간 응답 consensus 분석
- pseudo-label 생성
- uncertainty 추정
- noisy-label learning
- annotator agreement 기반 학습

SSAFY 기본 모델 자체의 supervised learning은 `train.csv`의 gold answer에 의존한다.

3.3 Test Dataset

`test.csv`에는 정답이 포함되지 않는다.

최종 모델은 각 test sample에 대해

```
a / b / c / d
```

중 하나를 출력하고 다음 형식의 submission을 생성한다.

```
id,answer
test_0001,a
test_0002,c
...
```

4. Data Validation and Reproducibility

SSAFY는 학습 전에 데이터 구조를 적극적으로 검사한다.

다음 항목을 검증한다.

- CSV 필수 column 존재 여부
- train 정답이 `a/b/c/d` 범위에 있는지 여부
- 이미지 파일이 실제 경로에 존재하는지 여부

이를 통해 긴 모델 학습 이후에 데이터 경로 오류가 발견되는 문제를 방지한다.

재현성 확보를 위해 다음 random seed를 고정한다.

```
SEED = 42

random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
```

따라서 동일한 데이터 및 실행 환경에서는 train/validation split과 training subset이 동일하게 재현된다.

5. Stratified Sampling

SSAFY는 전체 training pool을 사용하거나 제한된 수의 샘플만 사용할 수 있다.

`TRAIN_LIMIT` 이 지정되면 `a`, `b`, `c`, `d` 클래스에서 가능한 한 동일한 수를 추출한다.

예를 들어

```
TRAIN_LIMIT = 200
```

인 경우 기본적으로 다음과 같이 구성된다.

```
a ≈ 50  
b ≈ 50  
c ≈ 50  
d ≈ 50
```

이를 통해 작은 실험 데이터에서도 특정 answer position에 대한 심각한 class imbalance가 발생하는 것을 막는다.

또한 sampling 이후 전체 subset을 다시 shuffle한다.

6. Base Vision-Language Model

SSAFY는 Qwen-VL 계열의 instruction-tuned multimodal model을 backbone으로 사용한다.

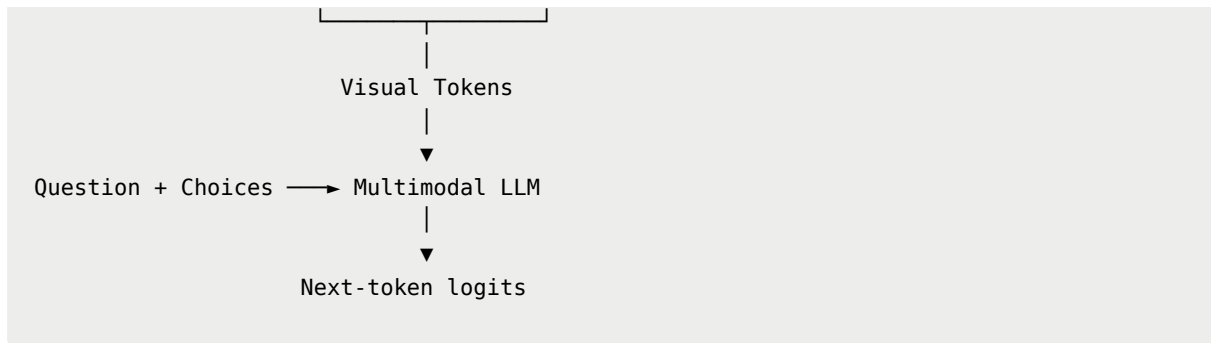
입력은 다음 두 modality로 구성된다.

```
Image  
+  
Question  
+  
Four Candidate Answers
```

모델 내부에서는 vision encoder가 이미지를 visual embedding으로 변환하고, language model은 visual token과 textual token을 함께 처리한다.

개념적인 구조는 다음과 같다.

```
Image → [ Vision Encoder ]
```



SSAFY는 이러한 pretrained multimodal representation을 유지하면서 특정 objective에 필요한 부분만 parameter-efficient fine-tuning한다.

7. 4-bit Quantization

GPU memory 사용량을 줄이기 위해 backbone model은 **4-bit NF4 quantization**으로 로드된다.

설정은 다음과 같다.

```

BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=COMPUTE_DTYPE,
)

```

NF4(NormalFloat 4)는 neural-network weight의 통계적 분포에 맞도록 설계된 4-bit representation이다.

SSAFY에서는 추가적으로 double quantization을 사용하여 quantization parameter 자체의 memory overhead도 줄인다.

개념적으로:

$$W_{FP} \rightarrow Q_{NF4}(W)$$

이며 base weight는 양자화된 상태로 유지된다.

8. QLoRA

SSAFY의 fine-tuning은 **QLoRA** 방식으로 수행된다.

QLoRA는 다음 두 아이디어의 결합이다.

Quantized Frozen Backbone
+
Trainable LoRA Adapter

원래 linear transformation이

$$h = Wx$$

라고 하면 LoRA는 이를

$$Wx + \Delta Wx$$

로 확장한다.

여기서

$$A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{d' \times r}$$

이다.

SSAFY의 rank는 8이며 주요 LoRA parameter는 다음과 같다.

r	= 8
alpha	= 16
dropout	= 0.05
bias	= none

9. Language-Only LoRA

SSAFY의 중요한 설계 선택 중 하나는 **vision encoder**를 직접 **fine-tuning**하지 않는 것이다.

LoRA target을 탐색할 때 module 이름에

```
vision  
visual
```

이 포함되어 있으면 명시적으로 제외한다.

대신 language model의 다음 projection에만 LoRA를 적용한다.

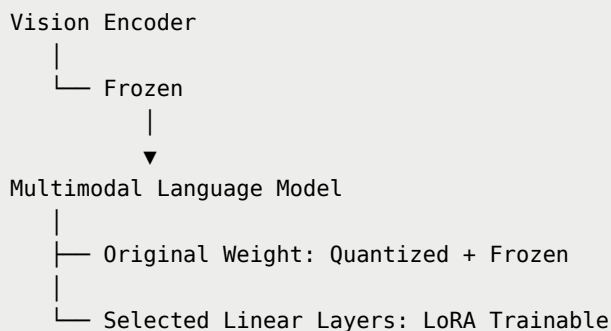
Self-Attention

```
q_proj  
k_proj  
v_proj  
o_proj
```

Feed-Forward Network

```
gate_proj  
up_proj  
down_proj
```

따라서 전체 구조는 다음과 같다.



이 설계는 세 가지 장점을 가진다.

첫째, GPU memory와 optimizer state를 크게 줄인다.

둘째, pretrained visual representation을 보존하여 작은 domain-specific dataset에서 vision encoder가 과적합되는 것을 방지한다.

셋째, VQA에서 중요한 이미지와 질문·선지 간 관계를 language-side attention과 MLP가 적응하도록 한다.

10. Gradient Checkpointing

SSAFY는 quantized model에 대해 gradient checkpointing을 활성화한다.

일반적인 backpropagation에서는 forward intermediate activation을 GPU memory에 저장한다.

Gradient checkpointing에서는 일부 activation을 저장하지 않고 backward 시 다시 계산한다.

즉:

```
Memory consumption ↓  
Computation time   ↑
```

의 trade-off가 발생한다.

대규모 VLM fine-tuning에서 제한된 VRAM으로 더 큰 이미지나 batch를 처리하기 위한 선택이다.

11. Prompt Design

각 sample은 multimodal chat 형태로 변환된다.

System prompt는 다음 목적을 가진다.

- 이미지를 분석할 것
- 객관식 문제임을 인식할 것
- 반드시 a/b/c/d 중 하나만 출력할 것
- 설명을 생성하지 않을 것

user input은 다음과 같은 구조다.

<IMAGE>

Question

- (a) Option A
- (b) Option B
- (c) Option C
- (d) Option D

정답을 a, b, c, d 중 하나의 소문자 한 글자로만 출력하세요.

학습 중에는 마지막에 gold assistant response를 추가한다.

assistant:
c

즉 하나의 training sequence는 개념적으로 다음과 같다.

```
System Instruction
  ↓
Image
  ↓
Question
  ↓
Choices
  ↓
Assistant Generation Boundary
  ↓
Gold Letter
```

12. Assistant-Only Supervision

SSAFY의 가장 중요한 학습 설계 중 하나는 **prompt 전체에 loss를 걸지 않는 것**이다.

일반적인 causal LM fine-tuning에서 전체 token에 label을 복사하면 모델은 다음 내용을 모두 학습하려 한다.

```
system prompt
question
candidate answers
assistant answer
```

그러나 VQA objective에서 필요한 것은 정답 prediction뿐이다.

따라서 SSAFY는 대부분의 token label을 -100으로 mask한다.

PyTorch CrossEntropyLoss에서는 **-100** 인 위치는 loss 계산에서 제외된다.

개념적으로:

```
Prompt token      → -100
Image-related token → -100
Question token     → -100
Choice token       → -100
Assistant "c"      → c-token ID
Closing token      → -100
```

결과적으로 supervised signal은 오직 정답에 해당하는 한 token에서만 발생한다.

13. Exact Answer Token Masking

단순히 문자열 길이나 token index를 추정하지 않고, SSAFY는 두 sequence를 별도로 tokenize한다.

Prompt-only sequence

```
system + user + assistant generation prefix
```

Full sequence

```
system + user + assistant answer
```

두 token sequence가 동일한 prefix를 가지는지 직접 확인한 뒤, prompt length에 해당하는 첫 assistant token만 label로 설정한다.

이는 tokenizer와 chat template 변화에 따른 잘못된 masking을 방지한다.

추가 sanity check를 통해 실제 supervised target을 decode하여 `a/b/c/d`가 포함되는지도 검증한다.

14. Training Objective

SSAFY 기본 버전의 training objective는 causal language modeling cross entropy 이다.

gold choice가 (y)이고 assistant answer 위치에서 모델이 생성한 vocabulary distribution을

$$p(v|I, Q, O)$$

라고 하면 loss는

$$-\log p(y|I, Q, O)$$

이다.

예를 들어 정답이 `c` 이고 $P(c) = 0.8$ 이면

$$-\log(0.8) \approx 0.223$$

이다.

중요한 점은 SSAFY 기본 버전에서는 `a/b/c/d` 네 개만을 대상으로 별도의 classifier를 학습하는 것이 아니라는 점이다.

학습 과정에서는 전체 vocabulary 중 gold letter token의 probability를 높이는 causal LM objective를 사용한다.

반면 inference에서는 네 choice token만 비교한다.

15. Gradient Accumulation

SSAFY는 limited VRAM에서 큰 effective batch size를 얻기 위해 gradient accumulation을 사용한다.

설정:

```
TRAIN_BATCH_SIZE = 8
GRAD_ACCUM       = 4
```

따라서 일반적인 effective batch size는 32이다.

즉 mini-batch 4개의 gradient를 누적한 후 한 번 optimizer update를 수행한다.

마지막 accumulation group의 batch 개수가 4보다 적은 경우에는 실제 group size를 계산하여 loss scaling을 보정한다.

이를 통해 마지막 incomplete accumulation group에서 gradient magnitude가 비정상적으로 작아지는 문제를 방지한다.

16. Optimizer

Optimizer는 AdamW를 사용한다.

```
Learning Rate = 1e-4
Weight Decay  = 0.01
```

AdamW update는 일반적으로 다음과 같은 형태를 갖는다.

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_{t-1}$$

기호	의미
θ	모델의 파라미터(weight)
θ_{t-1}	업데이트 전 파라미터
θ_t	업데이트 후 파라미터
t	학습의 현재 step
η	learning rate(학습률)
λ	weight decay 계수
m_t	gradient의 1차 모멘트 추정치
v_t	gradient의 2차 모멘트 추정치
$\hat{m}_t$	bias correction된 m_t
$\hat{v}_t$	bias correction된 v_t
ϵ	0으로 나누는 것을 막는 아주 작은 값
g_t	현재 step에서 계산된 gradient(기울기)

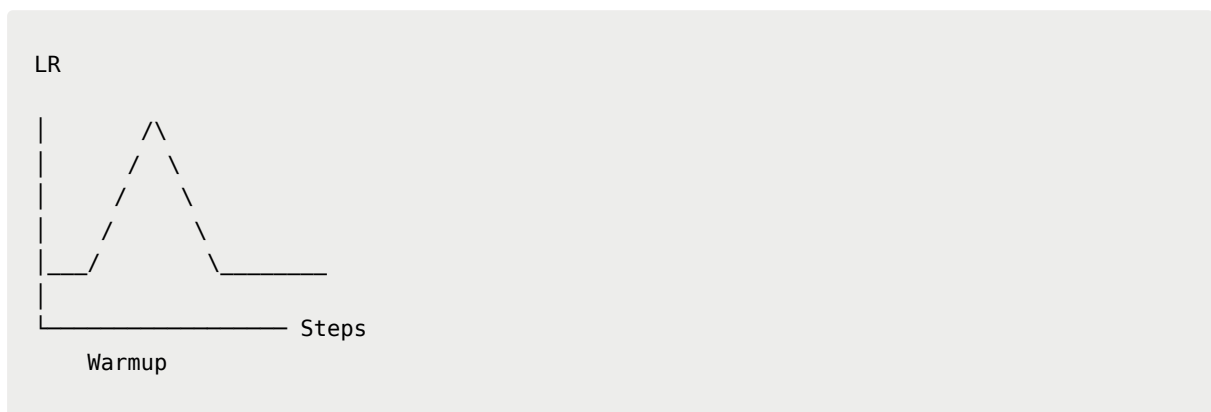
17. Learning-Rate Scheduling

SSAFY는 linear learning-rate scheduler와 warmup을 사용한다.

전체 optimizer step 수가 T라면 warmup step은 $0.03T$ 로 설정된다.

초기에는 learning rate를 점진적으로 증가시키고, 이후 training 종료 시점까지 선형 감소한다.

개념적으로:



초기 parameter update가 pretrained representation을 급격히 파괴하는 것을 완화하기 위한 목적이다.

18. Mixed-Precision Computation

GPU가 BF16을 지원하면 SSAFY는

```
torch.bfloat16
```

을 사용한다.

그렇지 않으면 FP16으로 fallback한다.

```
BF16 available
↓
BF16 autocast

BF16 unavailable
↓
FP16 autocast
↓
GradScaler
```

BF16은 FP32와 비슷한 exponent range를 가지므로 FP16보다 numerical stability가 높은 편이다.

FP16을 사용하는 경우에만 **GradScaler**를 활성화하여 gradient underflow를 방지한다.

19. Gradient Clipping

Optimizer update 전에 trainable parameter의 gradient norm을 제한한다.

즉

```
MAX_GRAD_NORM = 1.0
```

을 사용한다.

이는 특정 batch에서 큰 gradient가 발생하여 LoRA parameter가 급격하게 변경되는 것을 방지한다.

20. Validation Strategy

Training data의 10%는 validation set으로 고정된다.

```
train_test_split(
    train_df,
    test_size=0.1,
    random_state=42,
    stratify=train_df["answer"],
)
```

이 validation set은 학습 중 다시 training set으로 이동하지 않는다.

따라서 각 evaluation에서 동일한 sample을 사용하여 epoch 간 성능을 비교할 수 있다.

현재 설정에서는

```
NUM_EPOCHS = 20
EVAL_EVERY = 10
```

이므로 기본적으로 epoch 10과 epoch 20에서 validation accuracy를 계산한다.

21. Best-Checkpoint Selection

각 validation 시점에서 accuracy를 측정한다.

현재 best accuracy보다 높은 경우 해당 LoRA adapter state를 CPU memory에 복사하여 보관한다.

```
Epoch 10
Validation = 0.86
→ Best

Epoch 20
Validation = 0.84
→ reject

Final model
→ Epoch 10 adapter 복원
```

따라서 마지막 epoch의 모델을 무조건 사용하는 것이 아니라 **validation accuracy 기준 best checkpoint**를 최종 모델로 선택한다.

22. Direct Multiple-Choice Inference

학습 방식과 달리 inference에서는 자연어 generation을 수행하지 않는다.

먼저 chat template상에서 assistant가 `a`, `b`, `c`, `d`를 출력할 때 실제로 어떤 tokenizer ID가 사용되는지 동적으로 탐색한다.

즉 단순히

```
tokenizer.encode("a")
```

를 신뢰하지 않는다.

실제 Qwen chat template에서

```
assistant generation prefix + "a"
```

를 tokenize하고 generation prefix 이후 첫 token을 선택한다.

결과:

```
a → token_id_a
b → token_id_b
c → token_id_c
d → token_id_d
```

이 네 ID가 서로 다른지도 검사한다.

23. Logit-Based Choice Scoring

각 test sample에 대해 한 번 forward pass를 수행한다.

마지막 prompt position에서 vocabulary logits

$$z \in \mathbb{R}^{|V|}$$

를 얻는다.

그중 네 token만 선택한다.

$$s = [z_a, z_b, z_c, z_d]$$

그리고

$$\arg \max s$$

로 prediction을 결정한다.

예:

```
a = 3.21  
b = 1.94  
c = 7.12  
d = 0.63
```

이면

```
prediction = c
```

이다.

이 방식의 주요 장점은 생성 기반 inference에서 발생할 수 있는 다음 문제를 제거한다는 것이다.

```
"정답은 c입니다."  
"C"  
"(c)"  
"C."  
"플라스틱입니다."
```

모두 생성 모델 관점에서는 가능한 응답이지만 competition evaluator는 단일 letter만 요구한다.

SSAFY는 처음부터 후보 token score만 비교하므로 이러한 output parsing ambiguity가 존재하지 않는다.

24. Training-Inference Relationship

SSAFY의 중요한 특징은 training과 inference objective가 비슷하지만 완전히 동일하지는 않다는 점이다.

Training

$$P(y|\text{entire vocabulary})$$

를 최대화한다.

즉 정답 **c** 라면 vocabulary 전체에 대해 **c** token의 probability를 높인다.

Inference

$$\arg \max z_a, z_b, z_c, z_d$$

만 계산한다.

즉 전체 vocabulary 중 다른 단어의 logit은 최종 decision에 영향을 주지 않는다.

이 구조는 기본 SSAFY 모델의 단순성과 안정성을 확보하지만, 후속 모델인 SSAFY-T에서는 이 차이를 줄이기 위해 보다 직접적인 multiple-choice objective를 사용할 수 있다.

25. Prediction Diagnostics

SSAFY는 최종 answer뿐 아니라 네 선택지 각각의 raw logit도 저장한다.

```
id
pred
logit_a
logit_b
logit_c
logit_d
```

따라서 단순 accuracy 외에도 다음을 분석할 수 있다.

Confidence margin

margin이 작으면 모델이 두 답 사이에서 불확실한 상황으로 볼 수 있다.

Prediction distribution

```
a : 24.7%
b : 25.1%
c : 26.0%
d : 24.2%
```

와 같이 특정 선택지로 prediction이 편향되는지 확인할 수 있다.

Error analysis

validation prediction을

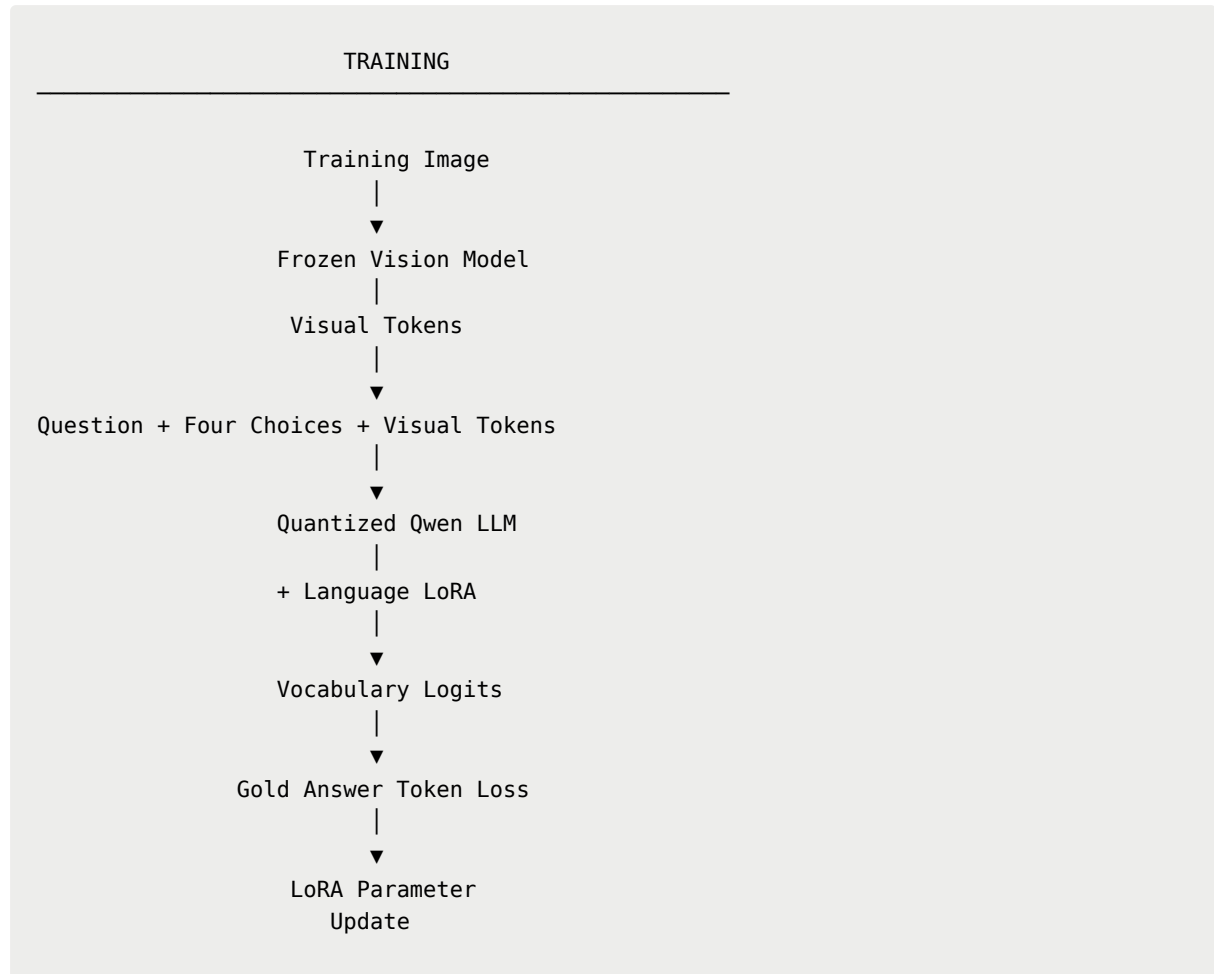
```
gold
pred
```

correct

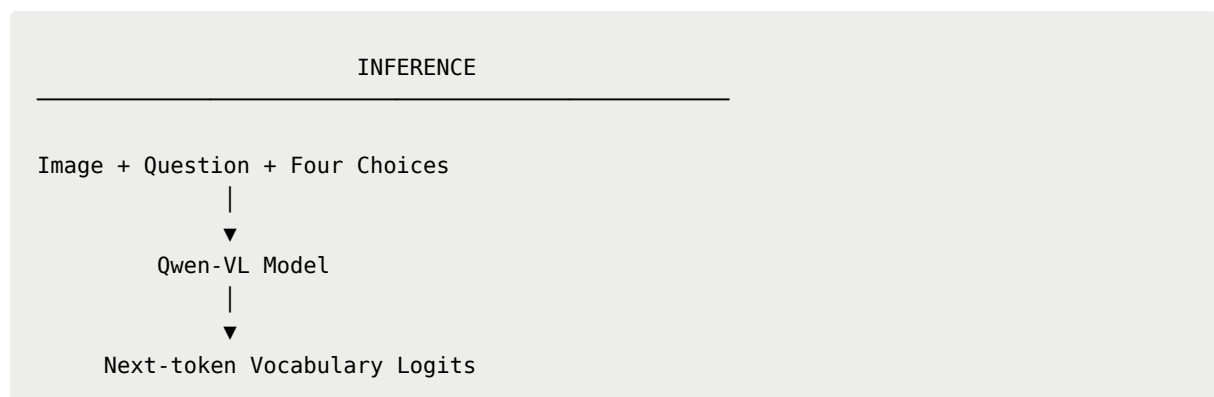
형태로 저장하므로 오답 sample을 별도로 분석할 수 있다.

26. Overall Architecture

SSAFY의 전체 training pipeline은 다음과 같이 정리할 수 있다.



Inference pipeline은 다음과 같다.





27. Parameter-Efficiency

SSAFY의 핵심 장점은 backbone 전체를 업데이트하지 않는다는 점이다.

Full fine-tuning의 경우 모든 parameter에 대해 다음 데이터가 필요하다.

```
Weight
Gradient
Optimizer first moment
Optimizer second moment
```

반면 SSAFY에서는 대부분의 backbone이

```
4-bit Quantized
+
Frozen
```

상태이고 LoRA parameter만 trainable하다.

따라서 full fine-tuning 대비 GPU memory 요구량과 optimizer state 크기를 크게 줄일 수 있다.

이는 제한된 단일 GPU 환경에서 수십억 parameter 규모의 VLM을 domain-specific VQA 문제에 적용할 수 있게 한다.

28. Why SSAFY Uses a Generative VLM for Classification

본 문제는 최종 출력만 보면 단순한 4-class classification으로 볼 수 있다.

그러나 각 선택지의 의미는 문제마다 달라진다.

예를 들어 **c** 라는 class는 고정된 의미를 가지지 않는다.

```
Question 1
c = 플라스틱

Question 2
c = 유리

Question 3
c = 종이
```

따라서 일반적인 CNN의 네 class classifier와는 문제 구조가 다르다.

필요한 것은

```
이미지 이해
+
질문 이해
+
선택지 의미 이해
+
이미지와 선택지 간 semantic alignment
```

이다.

SSAFY는 pretrained VLM을 사용함으로써 이 네 과정을 하나의 multimodal architecture에서 수행한다.

29. Design Philosophy

SSAFY의 기본 설계 철학은 **"Generate less, compare more."**로 요약할 수 있다.

일반 VLM은 자연어 answer generation에 최적화되어 있지만 본 task에서 요구되는 것은 네 선택지 중 하나의 선택이다.

따라서 SSAFY는 VLM의 풍부한 내부 표현 능력을 유지하되, 최종 출력 단계에서는 생성의 자유도를 강하게 제한한다.

이를 통해 다음을 동시에 얻는다.

- pretrained VLM의 visual reasoning 능력
- natural language question understanding

- semantic candidate comparison
 - deterministic output
 - low inference overhead
 - parsing error 제거
 - parameter-efficient training
-

30. Limitations

SSAFY 기본 모델에는 몇 가지 명확한 한계가 존재한다.

30.1 Limited Vision Adaptation

vision encoder는 완전히 frozen되어 있다.

따라서 일반적인 pretrained visual representation과 재활용품 domain 사이에 domain gap이 존재한다면 이를 직접 수정하기 어렵다.

30.2 Position Bias

정답은 항상 `a/b/c/d` token으로 학습되므로 모델이 선택지 의미 외에 answer-position prior를 학습할 가능성이 있다.

예를 들어 training data에서 `c`가 상대적으로 많이 등장한다면 모델이 이미지 evidence보다 `c` token 자체에 높은 prior를 형성할 수 있다.

30.3 LM Objective vs MC Objective

Training은 전체 vocabulary 기반 cross entropy이지만 evaluation은 네 choice token만 비교한다.

따라서 training objective와 competition metric 사이에 약간의 mismatch가 존재한다.

30.4 No Explicit Hard-Example Mechanism

모든 training sample은 DataLoader shuffle 이후 기본적으로 동일한 중요도로 처리된다.

모델이 지속적으로 틀리는 어려운 example을 특별히 다시 학습시키는 mechanism은 없다.

30.5 Single-Pass Inference

모든 sample은 동일한 computational budget으로 처리된다.

쉬운 문제와 어려운 문제 모두 단일 forward pass만 사용하며 모델 confidence에 따른 adaptive reasoning mechanism은 존재하지 않는다.

30.6 Limited Training Dataset in Experimental Configuration

코드의 현재 configuration에서는 다음 선언이 뒤에서 다시 적용된다.

```
TRAIN_LIMIT = 200
```

따라서 상단의

```
TRAIN_LIMIT = None
```

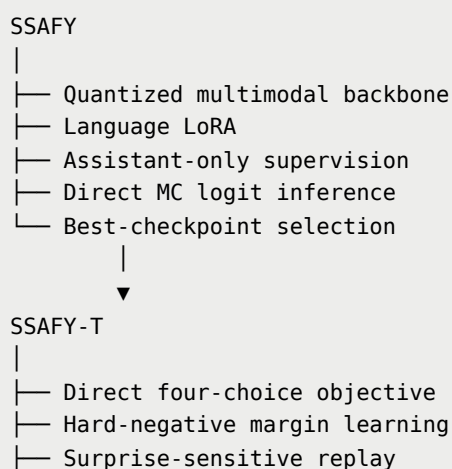
설정은 덮어써지고 실제 실행에서는 training pool 전체가 아니라 **200개만 사용된다**.

이는 모델 architecture 자체의 제한이라기보다 해당 실험 configuration의 제한이다.

31. Relationship to SSAFY-T

SSAFY는 후속 architecture인 **SSAFY-T (Surprise-Sensitive Adaptive Framework for Your Answers with Titans)**의 baseline이 된다.

개념적으로 발전 과정은 다음과 같다.



- └ Vision adaptation
- └ Pseudo-label learning
- └ Option permutation
- └ Adaptive inference
- └ Multi-model ensemble

따라서 SSAFY는 복잡한 adaptive mechanism을 추가하기 전, **QLoRA 기반 VLM을 객관식 VQA에 안정적으로 적용할 수 있는 최소 baseline architecture**의 역할을 한다.

32. Summary

SSAFY는 사전학습된 Vision-Language Model을 4-bit로 양자화하고 language-side LoRA만 학습하여 이미지 기반 객관식 질의응답 문제를 해결하는 parameter-efficient framework이다.

모델은 이미지와 질문, 네 선택지를 하나의 multimodal prompt로 처리하며, 학습 시 assistant 정답 token 하나에만 loss를 적용한다.

추론 시에는 자연어를 생성하지 않고 assistant generation boundary에서 **a**, **b**, **c**, **d** token의 logit을 직접 비교하여 최종 answer를 선택한다.

이를 통해 SSAFY는 다음 특성을 갖는다.

```

Pretrained Multimodal Knowledge
+
4-bit Quantization
+
Language-Side LoRA
+
Assistant-Only Supervision
+
Direct Choice Logit Scoring
↓
Efficient Multiple-Choice VQA

```

SSAFY의 목적은 최대한 복잡한 architecture를 구축하는 것이 아니라, **제한된 GPU 자원으로 pretrained multimodal model을 특정 VQA task에 효율적으로 적응시키고 안정적인 baseline을 구축하는 것이다.**

이 baseline은 이후 surprise-based adaptive learning, hard-negative optimization, vision adaptation 및 multi-stage inference를 도입하는 SSAFY-T의 출발점이 된다.